🤖 Git Helper — VS Code Chat Agent তৈরির নির্দেশিকা

> VS Code-এর Chat Panel-এ `@githelper` লিখলেই কাজ করবে।
> `@githelper clone`, `@githelper push`, `@githelper status` — এভাবে ব্যবহার করা যাবে।

📋 সূচিপত্র

1. প্রয়োজনীয় জিনিসপত্র
2. ফোল্ডার স্ট্রাকচার
3. ধাপ ১ — Extension প্রজেক্ট তৈরি
4. ধাপ ২ — package.json কনফিগার
5. ধাপ ৩ — extension.ts মূল কোড
6. ধাপ ৪ — gitService.ts সার্ভিস
7. ধাপ ৫ — envService.ts সার্ভিস
8. ধাপ ৬ — dbBackup.ts সার্ভিস
9. ধাপ ৭ — Build ও Run
10. ধাপ ৮ — VSIX প্যাকেজ বানানো
11. ব্যবহারের নিয়ম

প্রয়োজনীয় জিনিসপত্র

- ✅ Node.js v18+
- ✅ VS Code v1.90+ (Chat Participant API সাপোর্ট করে)
- ✅ Git ইনস্টল থাকতে হবে
- ✅ MySQL (যদি MySQL ব্যাকআপ চান)

```bash
# এই global প্যাকেজগুলো আগে ইনস্টল করুন
npm install -g yo generator-code vsce typescript
```

ফোল্ডার স্ট্রাকচার

```
githelper-agent/
│
├── src/
│   ├── extension.ts      ← মূল entry point, Chat Agent রেজিস্ট্রেশন
│   ├── gitService.ts     ← clone ও push লজিক
│   ├── envService.ts     ← .env পড়ার কাজ
│   └── dbBackup.ts       ← ডেটাবেজ ব্যাকআপ লজিক
│
├── package.json          ← Extension মেটাডেটা ও কনফিগ
├── tsconfig.json         ← TypeScript কনফিগ
└── .vscodeignore         ← প্যাকেজে কোন ফাইল বাদ যাবে
```

ধাপ ১ — Extension প্রজেক্ট তৈরি

টার্মিনালে চালান:

```bash
# Yeoman দিয়ে VS Code Extension স্ক্যাফোল্ড তৈরি
yo code
```

প্রশ্নের উত্তর এভাবে দিন:

```
? What type of extension do you want to create?
  → New Extension (TypeScript)          ✅ এটা বেছে নিন

? What's the name of your extension?
  → githelper-agent

? What's the identifier of your extension?
  → githelper-agent

? What's the description of your extension?
  → Git sync tool for office-home workflow

? Initialize a git repository?
  → Yes
```

```
? Which package manager to use?
  → npm
```

```bash
# তৈরি হওয়া ফোল্ডারে ঢুকুন
cd githelper-agent

# VS Code-এ খুলুন
code .
```

ধাপ ২ — package.json কনফিগার

`package.json` ফাইলটি সম্পূর্ণ মুছে নিচেরটি দিয়ে প্রতিস্থাপন করুন:

```json
```

```
{
 "name": "githelper-agent",
 "displayName": "Git Helper Agent",
 "description": "অফিস ও বাসার মধ্যে কোড sync করার VS Code Chat Agent",
 "version": "1.0.0",
 "engines": {
  "vscode": "^1.90.0"
 },
 "categories": ["Chat"],
 "activationEvents": [],
 "main": "./out/extension.js",
 "contributes": {
  "chatParticipants": [
   {
    "id": "githelper.agent",
    "name": "githelper",
    "fullName": "Git Helper",
    "description": "clone ও push দিয়ে অফিস-বাসা sync করুন",
    "isSticky": true,
    "commands": [
     {
      "name": "clone",
      "description": "GitHub থেকে সর্বশেষ ফাইল নামাও"
     },
     {
      "name": "push",
      "description": "DB ব্যাকআপসহ সব ফাইল GitHub-এ পাঠাও"
     },
     {
      "name": "status",
      "description": "বর্তমান git status দেখো"
     },
     {
      "name": "config",
      "description": ".env কনফিগারেশন দেখো"
     }
    ]
   }
  ],
  "configuration": {
   "title": "Git Helper Agent",
   "properties": {
    "githelper.envFilePath": {
```

```
        "type": "string",
        "default": "",
        "description": ".env ফাইলের পূর্ণ পথ"
      }
    }
  }
},
"scripts": {
  "vscode:prepublish": "npm run compile",
  "compile": "tsc -p ./",
  "watch": "tsc -watch -p ./"
},
"devDependencies": {
  "@types/node": "^20.0.0",
  "@types/vscode": "^1.90.0",
  "typescript": "^5.0.0"
}
}
```

ধাপ ৩ — extension.ts মূল কোড

`src/extension.ts` ফাইলটি সম্পূর্ণ মুছে নিচেরটি লিখুন:

```typescript
```

```
import * as vscode from "vscode";
import { GitService } from "./gitService";
import { EnvService } from "./envService";

// ── Agent ID ──────────────────────────────────────────────────────────────
const AGENT_ID = "githelper.agent";

export function activate(context: vscode.ExtensionContext) {
  console.log("✅ Git Helper Agent চালু হয়েছে");

  // ── Chat Participant তৈরি ──────────────────────────────────────────────
  const agent = vscode.chat.createChatParticipant(
    AGENT_ID,
    handleChatRequest
  );

  // Agent-এর আইকন সেট করুন (চাইলে একটি .png রাখুন)
  agent.iconPath = new vscode.ThemeIcon("git-merge");

  context.subscriptions.push(agent);
}

// ── মূল হ্যান্ডলার ──────────────────────────────────────────────────────
async function handleChatRequest(
  request: vscode.ChatRequest,
  context: vscode.ChatContext,
  stream: vscode.ChatResponseStream,
  token: vscode.CancellationToken
): Promise<void> {

  // workspace ফোল্ডার বের করো
  const workspaceFolder =
    vscode.workspace.workspaceFolders?.[0]?.uri.fsPath ?? "";

  if (!workspaceFolder) {
    stream.markdown(
      "❌ **কোনো workspace খোলা নেই।** আগে VS Code-এ প্রজেক্ট ফোল্ডার খুলুন।"
    );
    return;
  }

  const envService = new EnvService(workspaceFolder);
  const gitService = new GitService(workspaceFolder);
```

```
const cmd = request.command; // clone | push | status | config | undefined

// ── সাহায্য বার্তা (কোনো কমান্ড না দিলে) ──────────────────────────────────
if (!cmd) {
  stream.markdown(renderHelp());
  return;
}

// ── /clone ──────────────────────────────────────────────────────────────────
if (cmd === "clone") {
  stream.progress("GitHub থেকে ফাইল নামানো হচ্ছে...");
  try {
    const env = envService.load();
    const result = await gitService.clone(env);
    stream.markdown(result);
  } catch (err: any) {
    stream.markdown(`❌ **Clone ব্যর্থ হয়েছে:**\n\`\`\`\n${err.message}\n\`\`\`);
  }
  return;
}

// ── /push ───────────────────────────────────────────────────────────────────
if (cmd === "push") {
  const customMsg = request.prompt?.trim() || "";
  stream.progress("ডেটাবেজ ব্যাকআপ ও push চলছে...");
  try {
    const env = envService.load();
    const result = await gitService.push(env, customMsg);
    stream.markdown(result);
  } catch (err: any) {
    stream.markdown(`❌ **Push ব্যর্থ হয়েছে:**\n\`\`\`\n${err.message}\n\`\`\`);
  }
  return;
}

// ── /status ─────────────────────────────────────────────────────────────────
if (cmd === "status") {
  stream.progress("Git status চেক করা হচ্ছে...");
  try {
    const result = await gitService.status();
    stream.markdown(result);
  } catch (err: any) {
    stream.markdown(`❌ **Status দেখা যায়নি:**\n\`\`\`\n${err.message}\n\`\`\`);
  }
```

```
    return;
  }

  // ── /config ──────────────────────────────────────────────────────────────
  if (cmd === "config") {
    try {
      const env = envService.load();
      stream.markdown(renderConfig(env));
    } catch (err: any) {
      stream.markdown(`❌ **Config পড়া যায়নি:**\n\`\`\`\n${err.message}\n\`\`\`);
    }
    return;
  }
}

// ── UI Helper Functions ──────────────────────────────────────────────────
function renderHelp(): string {
  return `
## 🤖 Git Helper Agent

অফিস ও বাসার মধ্যে কোড sync করার টুল।

| কমান্ড | কাজ |
|---------|-----|
| \`@githelper /clone\` | GitHub থেকে সর্বশেষ ফাইল নামাও |
| \`@githelper /push\` | DB ব্যাকআপসহ সব ফাইল পাঠাও |
| \`@githelper /push লগইন পেজ শেষ\` | কাস্টম বার্তা দিয়ে push |
| \`@githelper /status\` | বর্তমান পরিস্থিতি দেখো |
| \`@githelper /config\` | .env কনফিগারেশন দেখো |

> ⚠️ আগে প্রজেক্টে \`.env\` ফাইল তৈরি করুন।
  `;
}

function renderConfig(env: Record<string, string>): string {
  return `
## ⚙️ বর্তমান কনফিগারেশন

| সেটিং | মান |
|---------|-----|
| REPO_URL | \`${env.REPO_URL || "সেট নেই"}\` |
| BRANCH | \`${env.BRANCH || "main"}\` |
| DB_TYPE | \`${env.DB_TYPE || "none"}\` |
| DB_NAME | \`${env.DB_NAME || "সেট নেই"}\` |
```

```
| PROJECT_DIR | \`${env.PROJECT_DIR || "./"}\` |
| BACKUP_DIR | \`${env.BACKUP_DIR || "./db_backups"}\` |

> 🔒 DB_PASS নিরাপত্তার জন্য দেখানো হয়নি।
  `;
}

export function deactivate() {}
```

ধাপ ৪ — gitService.ts সার্ভিস

নতুন ফাইল `src/gitService.ts` তৈরি করুন:

```typescript
```

```
import { execSync } from "child_process";
import * as fs from "fs";
import * as path from "path";
import { DbBackupService } from "./dbBackup";

export class GitService {
  constructor(private workDir: string) {}

  // ── Clone / Pull ──────────────────────────────────────────────────────────
  async clone(env: Record<string, string>): Promise<string> {
    const { REPO_URL, BRANCH = "main" } = env;

    if (!REPO_URL) {
      throw new Error(".env-এ REPO_URL সেট করা নেই।");
    }

    const hasGit = fs.existsSync(path.join(this.workDir, ".git"));
    let log = "";

    if (!hasGit) {
      // ── Fresh clone ──────────────────────────────────────────────────────
      log += "📦 প্রথমবার clone হচ্ছে...\n";
      const tmpDir = path.join(this.workDir, "__tmp_clone__");

      this.run(`git clone --branch ${BRANCH} ${REPO_URL} "${tmpDir}"`);

      fs.renameSync(
        path.join(tmpDir, ".git"),
        path.join(this.workDir, ".git")
      );

      for (const item of fs.readdirSync(tmpDir)) {
        fs.cpSync(
          path.join(tmpDir, item),
          path.join(this.workDir, item),
          { recursive: true, force: true }
        );
      }
      fs.rmSync(tmpDir, { recursive: true, force: true });
      log += "✅ Clone সম্পন্ন! সব ফাইল প্রস্তুত।";
    } else {
      // ── Pull ─────────────────────────────────────────────────────────────
      log += "🔁 রিপো পাওয়া গেছে, আপডেট টানা হচ্ছে...\n";
```

```
    this.tryRun("git stash");
    this.run(`git pull origin ${BRANCH} --rebase`);
    this.tryRun("git stash pop");
    log += "✅ সর্বশেষ আপডেট পাওয়া গেছে!";
  }

  return this.formatSuccess(log);
}

// ── Push ──────────────────────────────────────────────────────────────────
async push(env: Record<string, string>, customMsg: string): Promise<string> {
  const { BRANCH = "main", BACKUP_DIR = "./db_backups" } = env;
  let log = "";

  // 1. DB Backup
  const backupService = new DbBackupService(this.workDir);
  const backupResult = backupService.backup(env);
  log += backupResult + "\n";

  // 2. Git add
  this.run("git add -A");

  // 3. পরিবর্তন আছে কিনা চেক
  const statusOut = execSync("git status --porcelain", {
    cwd: this.workDir,
  }).toString().trim();

  if (!statusOut) {
    return this.formatInfo("ℹ️ কোনো পরিবর্তন নেই, push দরকার নেই।");
  }

  // 4. Commit
  const commitMsg = customMsg ||
    `sync: ${new Date().toLocaleString("bn-BD", { timeZone: "Asia/Dhaka" })}`;

  this.run(`git commit -m "${commitMsg}"`);

  // 5. Push
  this.run(`git push origin ${BRANCH}`);
  log += `\n✅ সব ফাইল GitHub-এ পাঠানো হয়েছে!\n📝 Commit: _${commitMsg}_`;

  return this.formatSuccess(log);
}
```

```
  // ── Status ──────────────────────────────────────────────────────────────
  async status(): Promise<string> {
    const hasGit = fs.existsSync(path.join(this.workDir, ".git"));
    if (!hasGit) {
      return this.formatInfo("⚠️ এই ফোল্ডারে এখনো git রিপো নেই।");
    }

    const branch = execSync("git branch --show-current", {
      cwd: this.workDir,
    }).toString().trim();

    const statusOut = execSync("git status --short", {
      cwd: this.workDir,
    }).toString().trim();

    const lastCommit = execSync(
      'git log -1 --pretty=format:"%h — %s — %cr"',
      { cwd: this.workDir }
    ).toString().trim();

    const changed = statusOut
      ? statusOut.split("\n").map((l) => `- \`${l}\``).join("\n")
      : "_কোনো পরিবর্তন নেই_";

    return `
## 📊 Git Status

| | |
|---|---|
| 🌿 Branch | \`${branch}\` |
| 🕐 শেষ Commit | ${lastCommit} |

### পরিবর্তিত ফাইল:
${changed}
    `;
  }

  // ── Internals ───────────────────────────────────────────────────────────
  private run(cmd: string): string {
    return execSync(cmd, { cwd: this.workDir }).toString();
  }

  private tryRun(cmd: string): void {
    try { this.run(cmd); } catch { /* উপেক্ষা */ }
```

```
  }

  private formatSuccess(msg: string): string {
    return `\n✅ **সফল!**\n\n${msg}\n`;
  }

  private formatInfo(msg: string): string {
    return `\nℹ️ ${msg}\n`;
  }
}
```

ধাপ ৫ — envService.ts সার্ভিস

নতুন ফাইল `src/envService.ts` তৈরি করুন:

```typescript
```

```typescript
import * as fs from "fs";
import * as path from "path";
import * as vscode from "vscode";

export class EnvService {
  private envPath: string;

  constructor(workDir: string) {
    // VS Code Settings-এ কাস্টম পথ না থাকলে workspace-এর .env ব্যবহার করো
    const customPath = vscode.workspace
      .getConfiguration("githelper")
      .get<string>("envFilePath");

    this.envPath = customPath
      ? path.resolve(customPath)
      : path.join(workDir, ".env");
  }

  load(): Record<string, string> {
    if (!fs.existsSync(this.envPath)) {
      throw new Error(
        `.env ফাইল পাওয়া যায়নি।\nখোঁজা হয়েছে: ${this.envPath}\n\n` +
        `.env ফাইল তৈরি করুন এবং REPO_URL সহ তথ্য পূরণ করুন।`
      );
    }

    const lines = fs.readFileSync(this.envPath, "utf-8").split("\n");
    const env: Record<string, string> = {};

    for (const line of lines) {
      const trimmed = line.trim();
      if (!trimmed || trimmed.startsWith("#")) continue;
      const eqIdx = trimmed.indexOf("=");
      if (eqIdx < 0) continue;
      const key = trimmed.slice(0, eqIdx).trim();
      const val = trimmed.slice(eqIdx + 1).trim().replace(/^["']|["']$/g, "");
      env[key] = val;
    }

    return env;
  }
}
```

ধাপ ৬ — dbBackup.ts সার্ভিস

নতুন ফাইল `src/dbBackup.ts` তৈরি করুন:

```typescript
```

```
import { execSync } from "child_process";
import * as fs from "fs";
import * as path from "path";

export class DbBackupService {
  constructor(private workDir: string) {}

  backup(env: Record<string, string>): string {
    const {
      DB_TYPE,
      DB_NAME,
      DB_USER,
      DB_PASS = "",
      DB_HOST = "localhost",
      SQLITE_PATH,
      BACKUP_DIR = "./db_backups",
    } = env;

    if (!DB_TYPE || DB_TYPE === "none") {
      return "ℹ️ ডেটাবেজ ব্যাকআপ স্কিপ (DB_TYPE=none)।";
    }

    const backupDir = path.resolve(this.workDir, BACKUP_DIR);
    if (!fs.existsSync(backupDir)) {
      fs.mkdirSync(backupDir, { recursive: true });
    }

    const ts = new Date()
      .toISOString()
      .replace(/[:.]/g, "-")
      .slice(0, 19);

    // ── MySQL ──────────────────────────────────────────────────────────────
    if (DB_TYPE === "mysql") {
      if (!DB_NAME || !DB_USER) {
        throw new Error("MySQL ব্যাকআপের জন্য DB_NAME ও DB_USER সেট করুন।");
      }
      const outFile = path.join(backupDir, `${DB_NAME}_${ts}.sql`);
      const passFlag = DB_PASS ? `-p${DB_PASS}` : "";
      execSync(
        `mysqldump -h ${DB_HOST} -u ${DB_USER} ${passFlag} ${DB_NAME} > "${outFile}"`
      );
      return `📦 MySQL ব্যাকআপ সম্পন্ন: \`${path.basename(outFile)}\``;
```

```
    }

    // ── SQLite ──────────────────────────────────────────────────────────
    if (DB_TYPE === "sqlite") {
      if (!SQLITE_PATH) {
        throw new Error("SQLite ব্যাকআপের জন্য SQLITE_PATH সেট করুন।");
      }
      const src = path.resolve(this.workDir, SQLITE_PATH);
      const outFile = path.join(backupDir, `db_${ts}.sqlite`);
      fs.copyFileSync(src, outFile);
      return `📦 SQLite ব্যাকআপ সম্পন্ন: \`${path.basename(outFile)}\``;
    }

    return `⚠️ অজানা DB_TYPE: ${DB_TYPE}`;
  }
}
```

ধাপ ৭ — Build ও Run

tsconfig.json নিশ্চিত করুন:

```json
{
  "compilerOptions": {
    "module": "commonjs",
    "target": "ES2020",
    "outDir": "out",
    "lib": ["ES2020"],
    "sourceMap": true,
    "rootDir": "src",
    "strict": true
  },
  "exclude": ["node_modules", ".vscode-test"]
}
```

Build করুন:

```bash
```

```
npm run compile
```

VS Code-এ টেস্ট করুন:

```
১. VS Code-এ F5 চাপুন
   → একটি নতুন "Extension Development Host" window খুলবে

২. সেই window-এ Ctrl+Shift+I চাপুন
   → Chat Panel খুলবে

৩. টাইপ করুন:
   @githelper /clone
   @githelper /push লগইন পেজ শেষ করলাম
   @githelper /status
   @githelper /config
```

ধাপ ৮ — VSIX প্যাকেজ বানানো

নিজের কম্পিউটারে ইনস্টল করার জন্য:

```bash
# প্যাকেজ বানান
vsce package

# ফাইল তৈরি হবে:
# githelper-agent-1.0.0.vsix

# VS Code-এ ইনস্টল করুন
code --install-extension githelper-agent-1.0.0.vsix
```

অথবা VS Code-এ:

```
Extensions → ··· (তিন বিন্দু) → Install from VSIX → ফাইল বেছে নিন
```

ব্যবহারের নিয়ম

প্রতিটি প্রজেক্টে একবার করুন:

প্রজেক্টের root-এ `.env` ফাইল তৈরি করুন:

```bash
REPO_URL=https://github.com/আপনার-username/আপনার-repo.git
BRANCH=main
DB_TYPE=mysql
DB_HOST=localhost
DB_USER=root
DB_PASS=your_password
DB_NAME=your_database
BACKUP_DIR=./db_backups
```

প্রতিদিনের ব্যবহার:

🏢 অফিসে কাজ শুরু: @githelper /clone
🏢 অফিসে কাজ শেষ: @githelper /push আজকের কাজ শেষ

🏠 বাসায় কাজ শুরু: @githelper /clone
🏠 বাসায় কাজ শেষ: @githelper /push রাতের কাজ সম্পন্ন

ফাইল তৈরির চেকলিস্ট

- ☐ src/extension.ts তৈরি করা হয়েছে
- ☐ src/gitService.ts তৈরি করা হয়েছে
- ☐ src/envService.ts তৈরি করা হয়েছে
- ☐ src/dbBackup.ts তৈরি করা হয়েছে
- ☐ package.json আপডেট করা হয়েছে
- ☐ tsconfig.json ঠিক আছে
- ☐ npm run compile সফলভাবে চলেছে
- ☐ F5 দিয়ে টেস্ট করা হয়েছে
- ☐ vsce package VSIX তৈরি হয়েছে

এই নির্দেশিকা অনুসরণ করে তৈরি Agent VS Code 1.90+ এ কাজ করবে।